

PYTHON PROJECT REPORT

Simple Grocery Shopping & Billing System

A command-line grocery billing project developed in Python using NumPy arrays

Project type	Console / command-line application
Primary purpose	Maintain a simple shopping list, calculate a bill, apply promotional codes, and display the final amount.
Language	Python
Library used	NumPy (imported with <code>from numpy import *</code>)

1. Project Overview

The submitted program is a simple grocery shopping and billing system intended to run in a terminal. The program first displays a list of grocery items and their displayed prices. It then accepts item names and quantities from the user, optionally accepts additional items, processes item-specific coupon codes, generates a promotional coupon when a 10-digit phone number is supplied, accepts a final coupon code, adds 5% tax, and prints the selected items with quantities and the calculated totals.

The implementation is compact and demonstrates several beginner-to-intermediate Python concepts: console input/output, loops, conditional branching, arrays, string processing, numeric calculations, and basic promotion logic.

Source basis: the analysis in this report is derived from the uploaded 93-line file `VitV8.py`.

Project Objectives

- Display the grocery items available in the store.
- Accept a list of purchased items and corresponding quantities.
- Support optional additions to the shopping list.
- Process item-specific coupon codes.
- Generate a coupon code after validating a 10-digit phone number.
- Apply a final coupon, calculate 5% tax, and display the grand total.
- Print each purchased item together with its quantity.

Expected User Interaction

1. Store list	2. Purchase input	3. Promotions	4. Billing
Program prints available items and displayed prices.	User enters item names and quantities.	Item-specific codes and a phone-based coupon are considered.	Subtotal is adjusted, 5% tax is added, and totals are printed.

2. Main Features and Components

Component	What the current code does	Relevant source
Store inventory	Defines a tuple containing eight displayed grocery items and prices.	Lines 3-6
Purchase input	Reads purchased item names and quantities and stores them in NumPy arrays.	Lines 7-14
Add items	Asks whether more items should be added, then merges additional item and quantity arrays.	Lines 18-25
Item coupons	Reads a number of coupon-code inputs and modifies selected values in the price array.	Lines 27-42
Bill calculation	Iterates through purchased items, matches names, and accumulates an amount using item-specific calculations.	Lines 43-64
Phone validation	Checks whether the entered phone number contains 10 characters. A coupon name is displayed when the check passes.	Lines 65-72
Final coupon	Accepts coupon input and applies percentage multipliers for recognized coupon names.	Lines 74-84
Tax and output	Adds 5% to the current amount, then prints items, quantities, subtotal, and grand total.	Lines 85-93

Displayed Inventory

Item	Displayed price
Bread	20 RS
Butter	50 RS
Cheese	80 RS
Chicken	300 RS
Chips	10 RS
Candy	5 RS
Toothpaste	40 RS
Chocolate	20 RS

The prices above are the values displayed by the program to the user. The billing section uses a separate NumPy array for calculation, so those displayed prices do not consistently match the later price references in the current implementation.

3. Program Workflow

The program follows a linear, prompt-driven sequence. The logic can be understood as the following processing pipeline:



Input Data Structures

Three arrays are central to the current design. `l1` holds the entered item names, `l2` holds quantities, and `l4` collects entered item-specific coupon codes. A separate array `l3` is used as a price/adjustment array during billing.

4. Pricing and Coupon Logic

The source contains two layers of promotion handling: item-specific codes and a final coupon entered after phone-number validation. The code applies percentage multipliers to the running amount before the final 5% tax calculation.

Coupon / trigger	Action in current code	Interpretation
riseup	Changes one element of the price array to half its current value.	Item-specific price adjustment.
clockadoodledo	Changes one element of the price array to two-thirds of its current value.	Item-specific price adjustment.
smoothlike	Changes one element of the price array to two-thirds of its current value.	Item-specific price adjustment.
10-character phone number	Prints the coupon code MONEYHEIST.	Coupon generation step.
money	Multiplies the current amount by 0.70.	30% reduction before tax, based on the code.
moneyheist	Multiplies the current amount by 0.80.	20% reduction before tax, based on the code.
father	Multiplies the current amount by 0.60.	40% reduction before tax, based on the code.

Tax calculation: the code sets $a1 = a * 0.05 + a$. Therefore, the grand total is calculated as the current amount plus 5% of that amount.

Conceptual Calculation

Stage	Formula represented in the code
Running amount	Start with $a = 0$, then add item-related amounts during the item loop.
Final coupon	Apply a multiplier such as 0.70, 0.80, or 0.60 when a matching code is entered.
Tax	$a1 = a + (0.05 * a)$
Grand total	Print $a1$ as the final total.

5. Technologies and Programming Concepts

Technology / concept	Use in the project
Python	Main programming language and runtime environment.
NumPy	Used to create and manipulate arrays for item names, quantities, prices, and coupon collections.
Console I/O	Uses <code>print()</code> for output and <code>input()</code> for interactive user entry.
Loops	Uses <code>for</code> loops to print inventory, process coupons, calculate items, and print quantities.
Conditional logic	Uses <code>if / elif / else</code> to recognize options, coupons, and item names.
String methods	The code attempts to use <code>lower</code> for case-insensitive matching.

Learning Outcomes Demonstrated

- Breaking a small business task into sequential input, processing, and output steps.
- Using arrays to store related values and iterate through them.
- Applying arithmetic formulas for discounts and tax.
- Building a basic interactive command-line interface.
- Using branching logic to implement business rules.

Suggested Project Use

This program is suitable as an educational project for demonstrating basic Python syntax, loops, arrays, user input, and arithmetic. A production-ready grocery billing application would normally use a more structured data model and stronger validation, but those features are outside the scope of the uploaded implementation.

6. Implementation Observations from the Submitted Code

The following points describe the code as written. They are included so the project report accurately reflects the current implementation rather than assuming behavior that is not present in the source.

Observation	Details
String comparison calls	Several conditions use expressions such as <code>b.lower=="add"</code> , <code>k.lower=="riseup"</code> , and similar checks for item and coupon names. In Python, <code>lower</code> is a method, so these comparisons do not invoke the method. The intended case-insensitive comparison appears to require calling the method.
Price array size / indexing	<code>l3</code> is created with six values, giving valid indices 0 through 5, while later code accesses <code>l3[6]</code> for chicken and for one item-specific coupon. Those execution paths can therefore raise an <code>IndexError</code> .
Displayed prices vs calculation array	The inventory text lists bread at 20 RS, butter at 50 RS, cheese at 80 RS, chicken at 300 RS, chips at 10 RS, candy at 5 RS, toothpaste at 40 RS, and chocolate at 20 RS. The later calculation uses separate positions in <code>l3</code> , and some positions do not correspond to those displayed values.
Extra loop iteration for coupon count	When a nonzero number <code>d</code> is entered, the loop uses <code>range(0, d+1)</code> . That iterates <code>d+1</code> times rather than <code>d</code> times, so the program asks for one more coupon entry than the entered count suggests.
Item list mismatch	The billing logic contains a branch for egg, but egg is not one of the eight items displayed in the store list.

Improvement Opportunities

- Replace parallel arrays with a dictionary or structured records mapping item names directly to prices.
- Normalize user input with `.lower()` before comparison and validate item names against the inventory.
- Validate that the number of quantities matches the number of items entered.
- Use a controlled coupon table so discount rates and coupon rules are defined in one place.
- Separate input, billing, discount, tax, and output logic into functions for easier testing and maintenance.
- Use explicit imports instead of `from numpy import *` to reduce namespace ambiguity.

7. Suggested Modular Design for a Future Version

A future version can keep the same overall project idea while making the program easier to read and test. The following logical modules map directly to the responsibilities already present in the source.

Function / module	Responsibility
show_inventory()	Display item names and current prices to the user.
read_cart()	Read item names and quantities, validate the entries, and build the cart.
apply_item_coupons()	Apply item-specific discounts before billing.
calculate_subtotal()	Calculate quantity x unit price for each cart item.
get_customer_coupon()	Validate phone input and obtain a coupon code.
apply_coupon()	Apply the selected percentage discount.
add_tax()	Compute the 5% tax amount and grand total.
print_bill()	Display item quantities, subtotal, tax-inclusive grand total, and closing line.

This modular structure is a design recommendation, not a claim about the existing source. The submitted file currently keeps the main logic in a single top-level script.

Project Summary

The uploaded program is a small command-line grocery billing project that combines inventory display, cart entry, coupon handling, discount arithmetic, tax calculation, and receipt-style output. It demonstrates core Python programming constructs and is suitable for a school or learning project. The implementation observations above should be considered when running or demonstrating the current version, because some code paths do not match the apparent intended behavior.



